

JAVASCRIPT · TEMA 01_2

Estructuras de control

Ramificaciones, bucles y captura de excepciones



```
if (aprendes) {  
  estructuras.dominar();  
} else {  
  seguir_practicando();  
}
```

00

Índice

- 2.1 Expresiones y sentencias
- 2.2 Ramificaciones (if / else / ?:)
- 2.3 Booleanos (truthy / falsy)
- 2.4 Comparación e igualdad estricta
- 2.5 Comparaciones mixtas (==)
- 2.6 Operadores booleanos (&& || ! ?? ?.)
- 2.7 La instrucción switch
- 2.8 Bucles while y do...while
- 2.9 Bucles for · for-of · for-in
- 2.10 break / continue / try-catch

2.1

Expresiones y sentencias

- Una expresión tiene un valor. Ej.: $6 * 7$ vale 42.
- Una sentencia (instrucción) no tiene valor: se ejecuta por su efecto.
- Sentencia de expresión = expresión + punto y coma.
- El punto y coma se inserta automáticamente al final de línea (ASI).

```
6 * 7;           // expresión: vale 42
                  // (el valor se descarta)

let number = 6 * 7; // sentencia: declara
                   // e inicializa

console.log(6 * 7); // sentencia de expresión
                   // con efecto secundario
```

2.2

Ramificaciones: if / else

- if (condición) instrucción — la condición va entre paréntesis.
- Para varias sentencias, usa un bloque { }.
- else es opcional; else if encadena varias condiciones.
- Estilo recomendado: "llave única" (la { al final de la línea).

```
if (yourSales > 2 * target) {  
    performance = 'Excelente';  
    bonus = 1000;  
} else if (yourSales > target) {  
    performance = 'Bueno';  
    bonus = 100;  
} else {  
    performance = 'Medio';  
    bonus = 0;  
}
```

2.2

El operador condicional ? :

- if es una sentencia: no puede usarse como expresión.
- condición ? primero : segundo — evalúa primero si se cumple la condición, y segundo si no.
- Útil para inicializar variables en una sola línea.
- Para el máximo/mínimo de valores, mejor usar Math.max() / Math.min().



//  error: if no es una expresión

```
let max = if (x > y) x else y;
```

//  operador ternario

```
let max = x > y ? x : y;
```

//  mejor aún, con la librería estándar

```
let max = Math.max(x, y);
```

2.3

Booleanos: valores "truthy" y "falsy"

- Las condiciones no tienen que ser booleanas: JavaScript las convierte.
- Valores "falsy": 0, NaN, null, undefined y la cadena vacía "".
- Todos los demás valores son "truthy".
- Regla de oro: conoce tus tipos y evita la conversión automática.



```
// ⚠️ peligroso: '' y 0 también son falsy
```

```
if (performance) { ... }
```

```
// ✅ claro: compara explícitamente
```

```
if (performance !== undefined) { ... }
```

Valores falsy

0

NaN

null

undefined

''

2.4

Comparación e igualdad estricta

- < <= > >= comparan números o cadenas (orden lexicográfico).
- Cualquier comparación con NaN da siempre false.
- === / !== → igualdad estricta: sin conversión de tipos.
- Para comprobar NaN, usa Number.isNaN(x), nunca x === NaN.

```
3 < 4 // true
'Hello' < 'Hi' // true
NaN >= 4 // false

'42' === 42 // false (tipos distintos)
'42' === '4' + '2' // true (mismo string)

Number.isNaN(x) //  forma correcta
```

2.5

Comparaciones mixtas: evita == y !=

- == y != permiten comparar tipos distintos convirtiéndolos.
- Las reglas de conversión son poco intuitivas y causan errores sutiles.
- Excepción tolerada: x == null comprueba null o undefined a la vez.
- Regla de oro n.º 3: usa siempre === y !==.

```
' ' == 0 // true 😞
'0' == 0 // true 😞
'0' == false // true 😞
' ' == '0' // false 😞

x == null // ✅ único uso tolerado:
           // x es undefined o null
```

2.6

Operadores booleanos: && || ! ?? ?.

- && y || se evalúan de forma perezosa (cortocircuito).
- ?? devuelve el valor por defecto solo si x es null o undefined.
- ?. accede a una propiedad de forma segura (evita excepciones).
- ?. se puede encadenar: person?.name?.length

```
// ⚠ falla si arg es 0, '' o false
let r = arg && arg.someMethod();

// ✅ solo usa el valor por defecto si es
// null o undefined
let v = arg.someMethod() ?? valorDefecto;

let recipient = person?.name;
let len = person?.name?.length;
```

2.7

La instrucción switch

- Compara una expresión con varios valores (case), con === (igualdad estricta).
- La ejecución continúa hasta encontrar un break o el final del switch.
- Sin break, el flujo "cae" al siguiente caso (fall-through).
- default se ejecuta si ninguna etiqueta coincide.

```
switch (someExpression) {  
  case 0:  
    description = 'zero';  
    break;  
  case false:  
  case true:  
    description = 'boolean';  
    break;  
  default:  
    description = 'other';  
}
```

2.8

Bucles while y do...while

- while (condición) instrucción — comprueba antes de cada vuelta.
- Si la condición es falsa desde el inicio, el cuerpo no se ejecuta nunca.
- do instrucción while (condición) — comprueba después: se ejecuta al menos una vez.
- do...while es mucho menos frecuente que while.



```
let years = 0;
while (balance < goal) {
  balance += paymentAmount;
  years++;
}
```



```
do {
  i++;
} while (i < s.length && s[i] !== ' ');
```

2.9.1

El bucle for clásico

- for (inicialización; condición; actualización) instrucción
- 1ª ranura: inicializa el contador.
- 2ª ranura: condición comprobada antes de cada iteración.
- 3ª ranura: actualiza el contador tras cada vuelta.

```
for (let i = 1; i <= 10; i++) {  
  console.log(i);  
}  
  
// recorrido en orden inverso  
for (let i = a.length - 1; i >= 0; i--) {  
  console.log(a[i]);  
}
```

2.9.2-3

Bucles for...of y for...in

- for...of → recorre los VALORES de un iterable
 - arrays, strings... en orden ascendente
- for...in → recorre las CLAVES de un objeto
 - orden no garantizado; también sirve para índices de un array



```
const arr = [10, 20, 30];
for (const element of arr) {
  console.log(element);
}
```



```
const obj = { name: 'Harry', age: 42 };
for (const key in obj) {
  console.log(`${key}: ${obj[key]}`);
}
```

2.10

Interrupción y continuación: break / continue

- break sale inmediatamente del bucle.
- continue salta al final de la iteración actual (a la siguiente vuelta).
- Una etiqueta (label:) antes de un bucle permite un break/continue etiquetado, útil en bucles anidados.
- Ambas son fáciles de evitar, pero conviene reconocerlas.

```
for (let i = 0; i < arr.length; i++) {  
  if (arr[i] <= 0) continue; // siguiente  
  if (arr[i] > 100) break;    // salir  
  sum += arr[i];  
}  
  
outer: while (i < arr.length) {  
  while (j < arr[i].length) {  
    if (arr[i][j] < 0) break outer;  
    j++;  
  }  
}
```

2.11

Captura de excepciones: try / catch

- Un método puede lanzar una excepción en vez de devolver un valor de error.
- try ejecuta el código; si algo lanza una excepción, salta a catch.
- Si no se captura ninguna excepción, el programa finaliza.
- Si no se produce ninguna excepción, catch nunca se ejecuta.

```
try {  
  let data = JSON.parse(input);  
  // si llega aquí, input era JSON válido  
  // ...procesar datos  
} catch {  
  // input no es JSON válido  
  console.log('Error en los datos');  
}
```

EN RESUMEN

Tres bloques para controlar el flujo

Ramificaciones

if / else · operador ?: · switch

Bucles

while · do-while · for · for-of · for-in

Excepciones

try / catch para errores en tiempo de ejecución

Regla de oro: conoce tus tipos y evita la conversión automática — usa `===` y `!==`, nunca `==` / `!=`.